Macro Pad Design

Noah Greskiewicz | ngreskie.github.io

Background

I was initially introduced to macro pads during the summer before my senior year of high school by a friend who was building one from scratch. Ever since, I hoped to eventually follow in his footsteps and built my own. This project covers the circuit design of keyboard matrices, PCB layout and routing, and the Python code required to enable the distillation of any combination of keyboard operations into one macro pad keystroke. If you follow along, by the end you will have built a custom 3x3 keyboard to speed up any repetitive keyboard process.

What is a Macro Pad?

A macro pad is an auxiliary human interface device (HID) that typically supplements a full QWERTY keyboard. Macro is short for macro instruction, which is a term that dates back to the 1950s and means a single command that triggers a sequence of actions automatically. A macro pad is a combination of button or other inputs that each i nitiate a unique macro. Modern uses for macro pads include extending a keyboard by including operations that are not available (buttons for screen casting, etc), or combining two or more keys into one ie. Control/ Command + C, used to copy information to a device clipboard. Macro pads may also include more inputs than simple switches/buttons, many feature dials/knobs, screens, etc.

Parts List

- Raspberry Pi Pico 2: The controller used for this project. It scans the key matrix, detects button presses, and uses CircuitPython HID libraries used to send keystroke inputs to the host device.
- Diodes: Placed in series with the switches, these are used in the key matrix to prevent ghosting when multiple keys are pressed.
- Kailh Mechanical MX Switches: The switches that the user physically presses to run a macro pad function. MX-compatible switches were selected because they are widely available, reliable, and compatible with a large variety of keycaps.
- Keycaps: The surface mounted to the switches. I purchase these for better durability and finish compared with 3D printed keycaps. However, a good DIY alternative is casting the keycaps in resin using silicon molds.
- PCB (Printed Circuit Board): The printed circuit board was designed from an initial schematic and connects the switches and diodes to the Pico via a matrix. Since a 3x3 board is a relatively small number of switches, it is also possible to connect components with wire instead of using a PCB; the latter sure does make your life easier though!

Bill of Materials

Quantity				Description	DigiKey Part Number	Cost
1				Macro Pad	.	$22.94
.	1			Enclosure - Top	.	$0.22
.	1			Enclosure - Bottom	.	$0.34
.	.	1		PCB	.	$6.11
.	.	.	1	Raspberry Pi Pico 2	2648-SC1631CT-ND	$6.12
.	.	.	9	Cherry MX Switches	1528-4955-ND	$0.7
.	.	.	9	Diodes	1N4148FS-ND	$0.04
.	.	.	9	Keycaps	.	$0.37
.	.	.	4	M3x10mm Bolts	.	$0.04

Circuit Design & Keyboard Matrix

For my 3x3 macro pad, I designed a circuit that uses a Raspberry Pi Pico 2 as a controller. To minimize the number of pins used on the Pico, I designed a keyboard matrix circuit. This is a common circuit used in keyboards as it allows a 3x3 keyboard to use only 6 pins on the controller compared to 9 pins if each key is connected independently. Lets build understanding one step at a time:

One Button:

A typical button would be be connected between a controller input pin and either V+ (3.3V) or GND (0V) (the controller input pin would be pulled DOWN or UP with a resistor, respectively). When the button is pressed, the initial/default state of the controller input pin (defined by pull UP/DOWN resistor) is flipped; this is sensed by the controller. When the button is released the controller pin returns to its default state. The button will **not** function if the input is pulled UP to 3.3V and the button is connected to 3.3V, or conversely, the input is pulled DOWN to GND and the button is connected to GND. For the button to be sensed by the controller, there must be a voltage difference across the button in its resting state.

Three Buttons (a single row):

For one row of three buttons, one side of each button is connected to V+ or GND; all three buttons are connected together on this side. The opposite side of each button is independently connected to its own input pin on the controller allowing for differentiation between each button. The controller listens for inputs on each input pin.

3x3 Matrix:

In both prior examples, the V+ or GND stayed constant. However, the full 3x3 matrix combines 3 rows, each connected to its own output pin on the controller. The output pin replaces V+ or GND. The positive consequence of this configuration is the selective control we now have over each row. When the rows are lined up, the columns of the matrix are created by connecting all buttons in the first position together, vertically. This creates column “A”, as

seen in Figure 1. Columns B and C are created in the same manor. Again, the columns are wired up vertically and the rows are connected horizontally, but the rows and columns can only be connected at a node when/where a button is pressed, otherwise the rows and columns are independent.

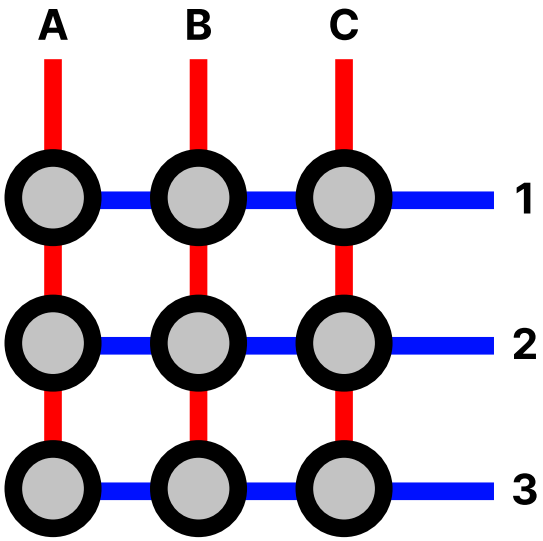

Figure 1: Key matrix example. Rows are blue and columns are red.

As mentioned in the last sentence of the One Button paragraph, there must be a voltage difference across the button. Using the Pico, we can set the outputs to either HIGH or LOW to isolate specific rows.

- Initial setting: The input pins connected to columns A, B, and C are pulled UP (3.3V). The rows 1, 2, and 3 are wired to the output pins on the Pico that are initially set to LOW (GND).
- As set up currently, because the columns are connected and all rows are set to LOW, if any button is pressed in column A the Pico registers it without knowing from which which row it came.
- To isolate a specific button in column A, first, rows 2 and 3 are set HIGH. This removes the voltage different across all buttons in those rows. Since only row 1 has a voltage difference between the inputs (columns) A, B, and C and output (row 1), this case becomes exactly the single row 3 button example. If any button in row 1 is pressed, the exact one is identified. Next, row 1 and 3 are set HIGH and 2 is set LOW. Rows 1 and 3 are "removed" and row 2 can be treated independently.
- With fast enough scanning and looping though the rows, isolating one at a time, the controller scans and registers independent keystrokes perfectly.

Ghosting:

When multiple keys are pressed at once, key ghosting can occur. This happens due to electrical current flowing back through the circuit. Looking at Figure 1, if we focus on an instant where row 2 is isolated and button 2B is pressed, the controller will read the input though column B. If button 1B is also pressed, the controller will not read it at this instantaneous moment; only

buttons on row 2 have a voltage difference from the Pico's input pins. However, if button 1B and 1A are pressed together, there is a direct path from column A and row 2. The controller will register this as button 2A, but only 2B, 1B, and 1A are pressed. With fast matrix scanning, the phantom press caused by ghosting will only last for an instant but is the source of much frustration.

A simple and inexpensive fix is to add diodes which limit the flow of current to only one direction. Using the above example but using Figure: 2 (which includes diodes), when column 2 is pulled DOWN to GND due to SW5 (2B) being pressed, compared row 1, it has a lower voltage. Consequently, if SW2 (1B) is pressed, the current cannot flow from row 1 into column because the diode stops the flow of current in that direction, and no ghosting occurs.

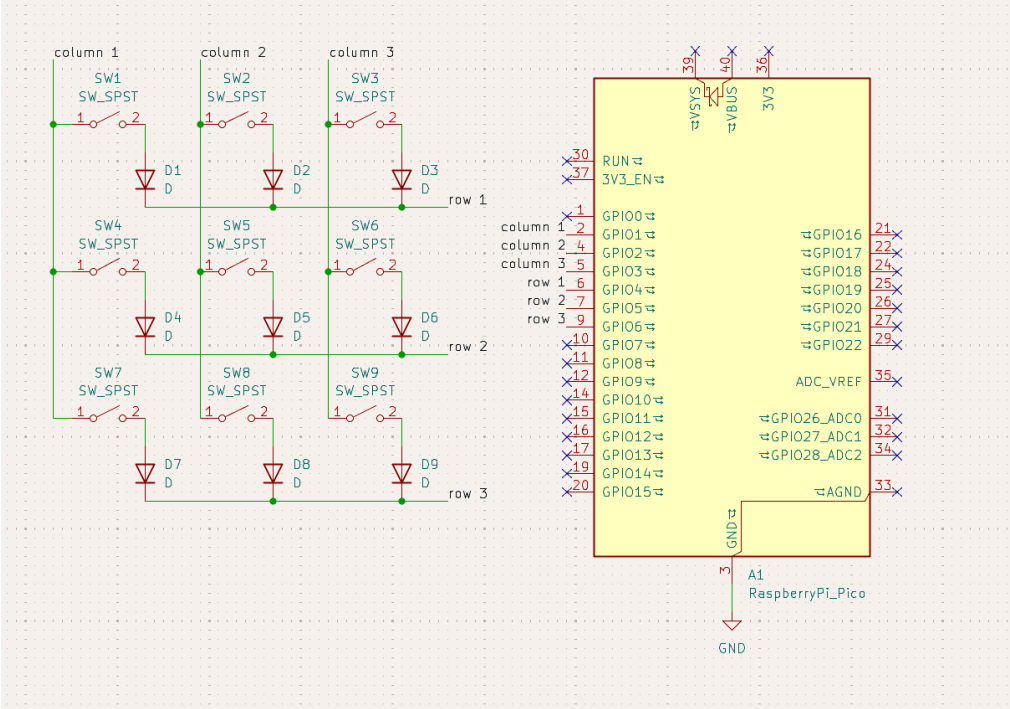

Figure 2: Key matrix schematic. Screenshot of KiCAD.

PCB Design

I used KiCAD to draw the circuit schematic, load component footprints, and generate gerber files for fabrication. After sketching the circuit in the schematic editor, the design was transformed into the PCB design side of KiCAD. At this point, I consulted Digikey's library of components to create a list of viable components to drive the footprint selection. While I choose popular diodes and controller (Pico), the Kailh mechanical switch footprints are not included in KiCAD by default; Fortunately, I found a GitHub repository that had footprint files. I began layout after all footprints were in the design. The switches are populated on a 19.05mm grid which ensures correct spacing when keycaps are added. Then, I positioned the Pico next to the switches and diodes as close as possible. I planned on using a trick during assembly to allow for the diodes to be placed with extreme proximity to the switches.

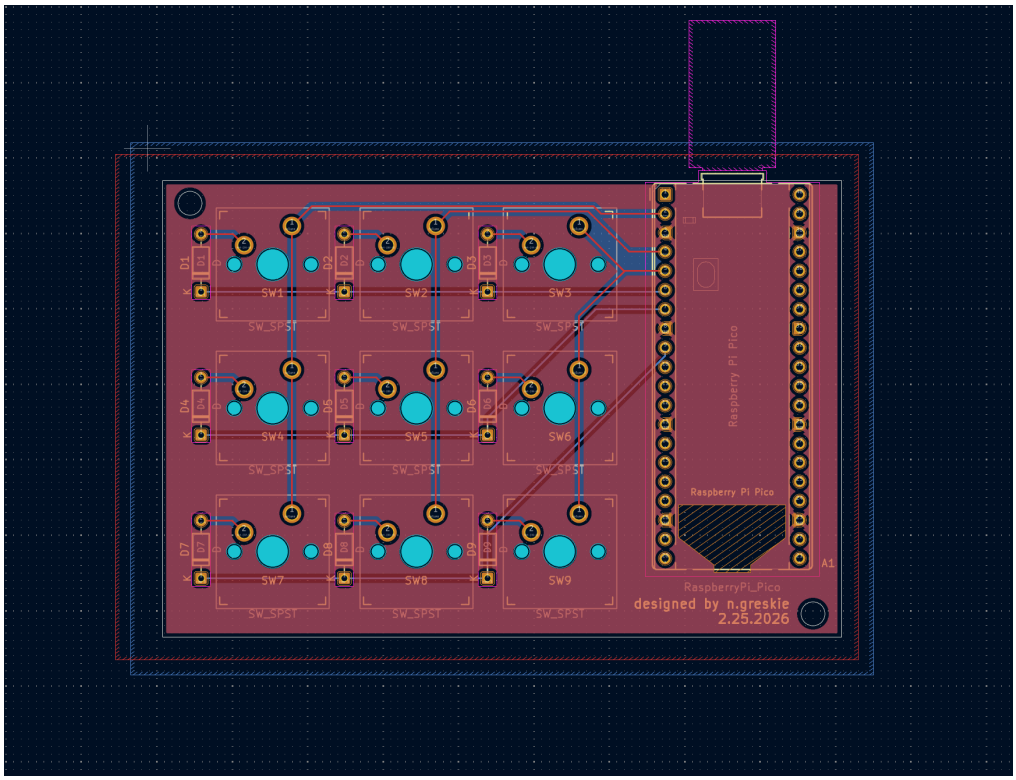

Figure 3: KiCAD PCB Editor view of final board layout.

PCB Assembly

I received five PCBs from JLC-PCB less than two weeks after placing my order. I tested the continuity of all traces before soldering and ensured all components fit into the drilled out footprints. The logic of the keyboard matrix I designed requires current to flow from columns to rows. This defines the direction of the diodes which must be oriented to match the KiCAD schematic. To test in which direction current can flow, I used a multimeter set to the diode test function. I decided to solder the diodes on the back of the PCB; while they fit when mounted to the front side, if a diode needs replacement, it will be much easier to solder and remove from the back without needing to pull all of the switches off as well.

When a switch is inserted into the through-hole footprint, its movement is not entirely constrained and there is a slight play in its positioning. To fix this, I designed a mid layer to tie all switches together that not only constrains small movements and makes soldering easier, it also supports the switches at another point aside from the solder joints on the PCB. For testing and software development purposes, I soldered wires to pin locations that the Pico would interface with on the PCB. After experimentation with the matrix code, I desoldered the test wires and replaced them with the Pico.

Enclosure Design

There were several features I wanted to include in the PCB enclosure design. I wanted the keys to be angled to support easier typing. This was achieved by mounting the PCB at the desired angle. Inside the enclosure, the PCB sits on angled ribs extruded from the bottom half of the shell. The top half has a large hole as to not interfere with the keycaps. M3 bolts are recessed

and are inserted from the bottom allowing for hidden hardware. When designing the PCB, I wanted the Pico to sit on the right side of the switches. I plan to use the macro pad with my left hand and the space on the top surface allows my thumb to rest above the Pico while typing. Another consideration while laying out footprints on the PCB was positioning the Pico's microUSB close to the edge of the board for easy cord access. When modeling the enclosure, I made the rear wall thin where the microUSB sits flush with the wall; this makes plugging a cord into the macro pad very easy.

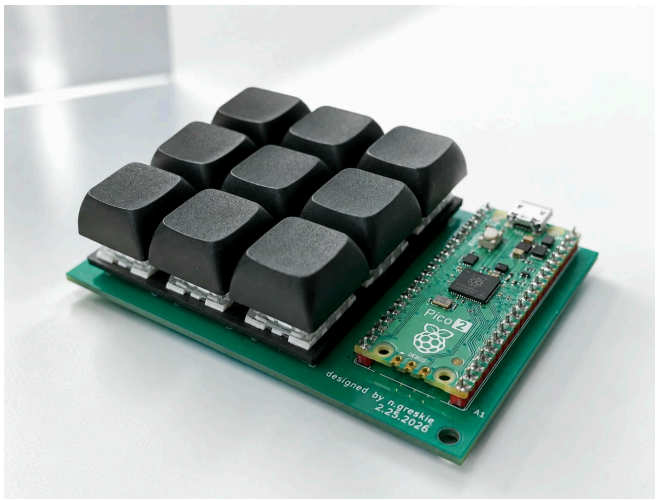

Figure 4: Assembled macro pad without enclosure.

Figure 5: SolidWorks macro pad assembly rendering without keycaps.

Software

- CircuitPython: The macro pad firmware is written in CircuitPython and runs directly on the Raspberry Pi Pico 2 CircuitPython was selected because it provides straightforward access to GPIO pins, USB HID functionality.
- Matrix scanning: As discussed in the circuit design section, the Pico scans the key matrix by pulling one row LOW at a time while reading the column inputs. This allows the controller to determine which button is pressed while minimizing the number of GPIO pins required.
- Edge detection: The matrix is scanned continuously at an approximate rate of 2000 (full matrix scans) per second. Without additional logic, holding a button down would cause its associated macro to execute repeatedly during every scan cycle. To prevent this behavior, the firmware stores the previous state of each key in a history dictionary. A macro is only executed when a key transitions from a released state to a pressed state (a falling-edge event). This approach ensures that each physical button press results in a single macro execution, regardless of how long the button remains held.
- Key mapping: Button actions are stored in a dictionary that maps each matrix position to a corresponding macro function. Separating the scanning logic from the macro definitions simplifies maintenance and makes it easy to change button assignments without modifying the matrix scanning code.

- HID libraries: CircuitPython's HID libraries provide access to several categories of input devices. The Keyboard library is used to send key combinations such as Ctrl+A, Ctrl+C, Ctrl+V, and Ctrl+W. The Consumer Control library is used for multimedia functions such as volume adjustment. These libraries abstract the underlying USB protocol.
- Types of operations:
  - Keyboard shortcuts (copy, paste, select all, close tab, etc.)
  - Consumer control commands (volume up and volume down)
  - Text string output
- Future software additions:
  - Operating-system detection to automatically use Command shortcuts on macOS and Control shortcuts on Windows/Linux
  - Option to hide the CircuitPython drive and source files during normal operation
  - External configuration software for key remapping
  - Additional layers or profiles that can be switched without reprogramming the device
  - Software debounce implementation

Final Thoughts

Overall, I am satisfied with the result of this project. However, in future iterations I want to build on the current design by adding a rotary encoder for volume control and a switch (ideally 3-position) that would enable multi-layer functionality. These additions, along with a resin-cast, machined, or injection molded case would catapult the product to the next level.

Figure 6: Final macro pad photo, $\frac{3}{4}$ view.